

Introduction à **Python** pour la Maintenance Électromécanique et l'Aérotechnique

De la théorie à la pratique avec Anaconda et Jupyter Notebook

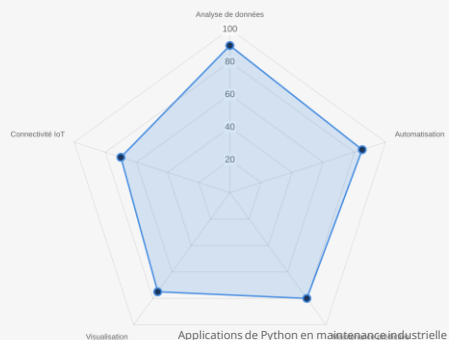


Manus AI, sous les ordres du Prof Papy NDUNGIDI

1

Python : Un atout pour le technicien moderne

-  **Langage simple et lisible** : Syntaxe claire et intuitive, idéale pour les débutants en programmation.
-  **Analyse de données** : Traitement des données de capteurs pour la maintenance prédictive et la détection de tendances.
-  **Automatisation** : Génération de rapports, gestion des plannings de maintenance et interaction avec les systèmes GMAO.
-  **Machine Learning** : Prédiction des pannes d'équipements et identification d'anomalies dans les données opérationnelles.
-  **Visualisation** : Création de graphiques et tableaux de bord interactifs pour surveiller l'état des équipements.



2

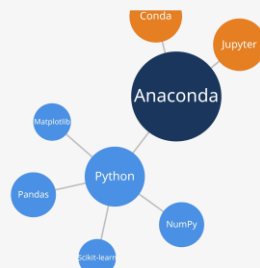
Anaconda : Votre boîte à outils tout-en-un

 **Distribution complète** : Python et plus de 1500 packages scientifiques préinstallés (NumPy, Pandas, Matplotlib, etc.).

 **Installation simplifiée** : Un seul installateur pour tous les composants, évitant les problèmes de dépendances.

 **Environnements virtuels** : Isolez vos projets avec des versions spécifiques de Python et de bibliothèques pour éviter les conflits.

 **Gestion de packages** : Installez, mettez à jour et supprimez facilement des bibliothèques avec Conda.



```
$ conda create --name maintenance python=3.9
$ conda activate maintenance
$ conda install pandas matplotlib scikit-learn
```

Commandes Conda essentielles pour la gestion d'environnements

3

Jupyter Notebook : Le laboratoire interactif

 **Environnement interactif** : Exécutez du code par cellules et visualisez immédiatement les résultats.

 **Cellules de code** : Écrivez et exécutez du code Python de manière modulaire et itérative.

 **Cellules Markdown** : Documentez votre code avec du texte formaté, des équations et des images.

 **Partage facilité** : Exportez vos notebooks en différents formats (HTML, PDF) pour les partager.

```
$ jupyter notebook
```

Markdown

Analyse de température d'un moteur

Calculons la température moyenne et identifions les valeurs dépassant le seuil d'alerte.

Code

```
import numpy as np

# Données de température
temperatures = [75.2, 76.1, 77.5, 78.9, 80.3, 81.5, 82.8]
```

```
# Calcul de la moyenne
moyenne = np.mean(temperatures)
print(f"Température moyenne: {moyenne:.2f}°C")
```

Température moyenne: 78.90°C

Code

```
# Détection des valeurs au-dessus du seuil
seuil = 80.0
alertes = [temp for temp in temperatures if temp > seuil]
print(f"Valeurs dépassant le seuil ({seuil}°C): {alertes}")
```

4

Variables : Stocker l'information

Qu'est-ce qu'une variable ?

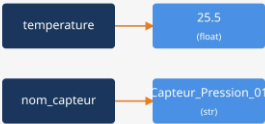
Une variable est un nom symbolique qui fait référence à une valeur stockée en mémoire, comme une boîte étiquetée contenant une information.

Déclaration et affectation

Python détecte automatiquement le type de la variable lors de l'affectation.

```
# Exemples de variables
temperature = 25.5
nom_capteur = "Capteur_Pression_01"
est_actif = True

# Affichage des valeurs
print(temperature) # Affiche: 25.5
```



Conventions de nommage

Utilisez **snake_case** pour les noms de variables en Python.

```
# Bonnes pratiques
niveau_huile = 80 # ✓ Descriptif snake_case
```

5

Types de Données : La nature de l'information

Nombres

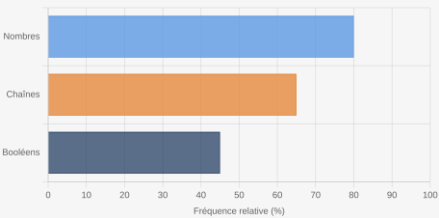
```
int (entiers): 10 -5
float (décimaux): 25.3 98.6
Exemple: temperature = 85.2
```

A Chaînes de caractères (str)

```
Texte entre guillemets: "Moteur_A" 'Alerte'
Concaténation: "Siemens" + " " + "S7-1500"
Formatage: f"Température: {temperature}°C"
```

• Booléens (bool)

```
Valeurs de vérité: True ou False
Exemple: est_en_marche = True
Utilisés pour les conditions et les décisions
```



Python détecte automatiquement le type des variables

6

Opérateurs Arithmétiques : Les calculs de base

Opérateur	Description	Exemple	Résultat
+	Addition	10 + 3	13
-	Soustraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3.33...
%	Modulo (reste)	10 % 3	1
**	Exposant	10 ** 3	1000

```
# Exemple de calcul de puissance électrique
tension = 230 # Volts
courant = 10 # Ampères

puissance = tension * courant

# Calcul de consommation d'énergie
duree = 2.5 # heures
energie = puissance * duree

print(f"Puissance : {puissance} W")
Puissance : 2300 W

print(f"Énergie consommée : {energie} Wh")
Énergie consommée : 5750 Wh
```



7

Décisions et Logique : Comparer et combiner

Opérateurs de comparaison

Opérateur	Description	Exemple	Résultat
==	Égal à	5 == 5	True
!=	Différent de	5 != 10	True
<	Inférieur à	5 < 10	True
>	Supérieur à	10 > 5	True

Opérateurs logiques

Opérateur	Description	Exemple	Résultat
and	ET logique	True and False	False
or	OU logique	True or False	True
not	NON logique	not True	False



```
# Application en maintenance
temperature = 85 # °C
pression = 10.2 # bar

# Vérification des seuils d'alerte
temp_ok = temperature < 90
press_ok = 9.5 <= pression <= 10.5

# Combinaison des conditions
systeme_ok = temp_ok and press_ok

print(f"État: {'Normal' if systeme_ok else 'Alerte'}")
```

8

Fonctions Mathématiques : Des outils avancés

Le module `math` offre des fonctions mathématiques essentielles pour les calculs d'ingénierie.

Constantes mathématiques

`math.pi` `math.e` `math.tau`

Fonctions algébriques

`math.sqrt()` `math.pow()` `math.log()` `math.log10()`

Fonctions trigonométriques

`math.sin()` `math.cos()` `math.tan()` `math.radians()`

Arrondis et valeurs absolues

`math.ceil()` `math.floor()` `math.fabs()`

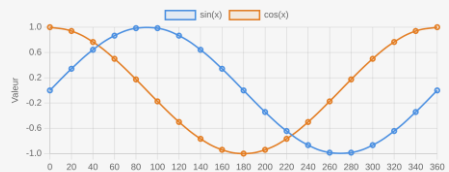
```
# Importation du module math
import math

# Calcul de l'aire d'un cercle
rayon = 5
aire_cercle = math.pi * (rayon ** 2)
```

```
# Calcul d'une racine carrée
valeur = 16
racine_carree = math.sqrt(valeur)
```

```
print(f"Aire du cercle : {aire_cercle:2f}")
Aire du cercle : 78.54
```

```
print(f"Racine carrée de {valeur} : {racine_carree}")
Racine carrée de 16 : 4.0
```



9

Listes : Organiser vos données en séquence

Qu'est-ce qu'une liste ?

Une collection ordonnée et modifiable d'éléments, pouvant contenir des types mixtes.

```
# Création d'une liste
equipements = ["Moteur_A", "Pompe_B", "Compresseur_C"]

# Accès par index (commence à 0)
premier = equipements[0] # "Moteur_A"
dernier = equipements[-1] # "Compresseur_C"
```

Méthodes principales

Méthode	Description	Exemple
<code>append()</code>	Ajoute à la fin	<code>liste.append(x)</code>
<code>insert()</code>	Insère à une position	<code>liste.insert(i, x)</code>
<code>remove()</code>	Supprime par valeur	<code>liste.remove(x)</code>
<code>sort()</code>	Trie la liste	<code>liste.sort()</code>

Représentation d'une liste Python



Application en maintenance

```
# Liste des relevés de température
tempertures_moteur = [78.5, 79.1, 80.2, 81.5, 82.0]
```

```
# Calcul de la moyenne
moyenne = sum(tempertures_moteur) / len(tempertures_moteur)
```

```
# Détection des valeurs au-dessus du seuil
seuil = 80.0
alertes = [temp for temp in tempertures_moteur if temp > seuil]
```

```
print(f"Température moyenne: {moyenne:1f}°C")
Température moyenne: 80.3°C
```

```
print(f"Alertes: {alertes}")
Alertes: [80.2, 81.5, 82.0]
```

10

Tuples : Données fixes et sécurisées

Qu'est-ce qu'un tuple ?

Une collection ordonnée et **immuable** d'éléments. Une fois créé, un tuple ne peut pas être modifié.

```
# Création d'un tuple
coordonnees = (10.5, 20.1, 5.0)
parametres_moteur = ("50kW", "400V", "80A")

# Accès par index (comme les listes)
puissance = parametres_moteur[0] # "50kW"
```



Comparaison avec les listes

Caractéristique	Tuples	Listes
Syntaxe	()	[]
Mutabilité	Immuable	Mutable
Performance	Plus rapide	Plus flexible
Cas d'usage	Données fixes	Collections modifiables

```
# Applications en maintenance électromécanique

# Coordonnées de capteurs (x, y, z)
capteurs = [
    ("Capteur_Temp", (10.5, 20.0, 5.0)),
    ("Capteur_Pression", (15.2, 18.5, 5.0))
]

# Paramètres de configuration
config_temperature = ("Température", 60.0, 90.0, "°C")

print(f"Plage: {config_temperature[1]}-{config_temperature[2]}")
print(f"{config_temperature[3]}")
# Plage: 60.0-90.0 °C
```

Dictionnaires : Accéder aux données par nom

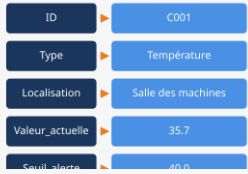
Qu'est-ce qu'un dictionnaire ?

Une collection non ordonnée de paires clé-valeur. Chaque clé est unique et immuable, associée à une valeur.

```
# Création d'un dictionnaire
capteur_info = {
    "ID": "C001",
    "Type": "Température",
    "Localisation": "Salle des machines",
    "Valeur_actuelle": 35.7,
    "Unite": "°C"
}

# Accès par clé
type_capteur = capteur_info["Type"]
print(type_capteur)
# Température
```

Structure d'un dictionnaire Python



Opérations courantes

```
# Ajout ou modification d'une paire clé-valeur
capteur_info["Seuil_alerte"] = 40.0

# Accès sécurisé avec .get()
statut = capteur_info.get("Statut", "Inconnu")
```

```
# Application en maintenance électromécanique

# Dictionnaire d'équipements
equipements = {
    "M001": {
        "type": "Moteur",
        "puissance": "75kW",
        "etat": "En service"
    },
    "P002": {
        "type": "Pompe",
```